

T-WEB PROJECT REPORT

"NeView"

Academic Year 2024/2025

Andrei Lucian Fistos (1060084)

Introduction

This report describes the development of the NeView web application, built as part of the T-WEB (6 CFU) course in the academic year 2024/2025.

NeView is a movie platform that combines data from two independent backend servers — one relational (PostgreSQL) and one document-based (MongoDB) — and presents them through a React single-page application. The platform allows users to browse the movie catalogue, read and write Rotten Tomatoes-style reviews, explore Oscar award history, track movie popularity, and chat in real time on each movie's page.

Technical Tasks — T-WEB

Solution Implemented

The application follows a three-service architecture, with each component running independently and communicating over HTTP and WebSocket:

- PostgreSQL Server — Spring Boot 3 / Java 21 (port 8080): serves the movie catalogue.
- MongoDB Server — Express.js / TypeScript / Node.js (port 4000): serves reviews, Oscar awards, popularity rankings, and user accounts.
- NeView Frontend — React 18 + Vite: the single-page application consumed by the browser.

The two backends are fully decoupled — the frontend holds two separate Axios instances, one per server, and orchestrates calls to both independently. There is no main server acting as a middleware between them; coordination happens client-side.

PostgreSQL Server (Spring Boot)

The PostgreSQL server contains the main movie dataset and exposes it through a REST API built with Spring Boot 3 and Spring Data JPA. The central entity is the movies table, which is the primary key anchor for all satellite tables: genres, actors, crew, posters, releases, themes, countries, studios, and languages.

Each entity follows the standard Spring layered pattern:

- An entity class (e.g. `Movie.java`) annotated with `@Entity`.
- A repository interface (e.g. `MovieRepository.java`) extending `JpaRepository`.
- A service class (e.g. `MovieService.java`) containing business logic and query orchestration.
- A controller class (e.g. `MovieController.java`) mapping HTTP verbs to service calls.
- One or more DTO records (e.g. `MovieDetailDTO.java`) defining the exact JSON response shape.

A key design decision was the introduction of a tiered loading model for movie card data, aimed at reducing initial homepage load time:

- Tier 1 — Slim (id, name, poster only): used for painting cards instantly.
- Tier 2 — Hover (genres, truncated description, rating): fetched on mouse-over with a 600 ms delay.
- Tier 2.5 — Expanded (full description + leading cast): fetched on chevron click.
- Tier 3 — Full detail (actors, crew, releases, languages, studios, themes): fetched only on the movie detail page.

The server also exposes batch endpoints that accept a list of movie titles and resolve them to card payloads in a single query — used to enrich popularity data (which only stores titles) with displayable fields from PostgreSQL.

MongoDB Server (Express.js)

The MongoDB server contains three datasets — Rotten Tomatoes-style reviews, Oscar award nominations, and user accounts — and exposes them through an Express.js REST API. Mongoose is used for schema definition and query execution.

Key architectural choices:

- Redis caching: all expensive aggregations are cached in Redis with TTLs tuned to data freshness needs (5 minutes for recent reviews, 24 hours for tomatometer stats, 90 days for Oscar data that changes only once a year).
- JWT authentication: user passwords are hashed with Argon2id via a pre-save hook. A signed JWT is issued on registration and login and validated by an Express middleware on every protected endpoint.
- Like synchronisation: like toggles are stored in MongoDB and asynchronously synced to the PostgreSQL likes counter via a BullMQ worker queue, ensuring the two databases stay consistent without blocking the API response.
- Real-time chat: Socket.IO provides per-movie chat rooms. Each room is identified as `movie-{title}`. Clients join and leave rooms explicitly via `joinMovie` and `leaveMovie` events.

- Popularity tracking: view counts and like events are written to Redis sorted sets and refreshed into daily and weekly rankings via scheduled cron jobs.

Frontend (NeView — React + Vite)

The frontend is a React 18 single-page application built with Vite. It uses React Router for client-side navigation across eight pages: Home, Films, Movie Detail, Search, Awards, Sign In, Sign Up, and Profile.

Key implementation decisions:

- Two Axios instances: one for the MongoDB server (with a JWT interceptor that attaches the stored token to every request) and one for the PostgreSQL server.
- AuthContext: a React Context backed by useReducer manages global authentication state. On mount it validates the stored token against GET /user/profile, clearing stale tokens automatically.
- useSocketChat hook: manages Socket.IO room membership per movie page. On mount it emits joinMovie; on unmount it emits leaveMovie so the server removes the socket from the room, preventing stale broadcast accumulation.
- Optimistic UI on likes: the LikeButton component updates the count instantly on click and rolls back if the backend returns an error or rate-limit response.

Requirements

- PostgreSQL database → PostgreSQL Server (Spring Boot JPA)
- MongoDB database → MongoDB Server (Mongoose)
- Express.js server → MongoDB Server
- Java Spring Boot JPA → PostgreSQL Server
- Socket.IO real-time chat → MongoDB Server + NeView frontend
- JWT authentication → MongoDB Server
- React single-page application → NeView (React + Vite)

Difficulties

- CORS configuration: the three services run on different ports (8080, 4000, and the Vite dev server), requiring explicit CORS policies on both backends.
- Tiered loading design: deciding how to split movie data across four tiers required careful measurement of payload sizes and perceived loading performance.
- Cache invalidation: choosing appropriate TTLs for Redis so that data feels fresh without hammering MongoDB on every request took iterative tuning.
- Like count consistency: keeping the MongoDB like toggle and the PostgreSQL persistent counter in sync without lost updates required the BullMQ async reconciliation approach.

Limitations

- Socket.IO authentication is not yet implemented — currently any connected client can join any chat room. JWT verification for WebSocket connections is planned.

- Three review endpoints (update, delete, like review) are stub implementations that return 501 Not Implemented.
- The MongoDB like toggle and like-status check are fully implemented on the server but not yet wired into the frontend LikeButton component.

Conclusions

Building NeView showed what full-stack development across heterogeneous data systems looks like in practice: the need to design clear contracts between services, the importance of caching for performance at scale, and how to handle consistency across two databases with different strengths.

The tiered loading model in particular required careful thinking about what data the user actually needs at each interaction stage, rather than fetching everything upfront. The use of AI tools helped accelerate debugging of server-side errors, especially around CORS, Redis configuration, and JWT middleware.

Work Division

Early stages of the project have been implemented along a colleague that soon after has given up on the project to pursue studies on other subjects. Everything else has been developed along Anthropic's Claude AI which has sped up the works by several times and taught a lot of strategies along this long adventure.

Extra Information

- A Redis instance must be running alongside the MongoDB server. It is used for response caching, the BullMQ like-sync queue, and per-user like cooldown keys.
- CORS: the MongoDB server allows requests from the Vite dev server origin; the PostgreSQL server allows requests from both the frontend and the MongoDB server.
- Environment variables: each service reads its configuration from a .env file (DB credentials, JWT secret, Redis URL, internal sync token). See the respective .env files in each service folder.

Bibliography

- Course material presented during lectures.
- Spring Boot 3 documentation — <https://spring.io/projects/spring-boot>
- Socket.IO documentation — <https://socket.io/docs>
- AsyncAPI specification — <https://www.asyncapi.com>
- Use of AI tools for understanding server-side errors and adapting code fragments.